SentinelPay Project Flow

Purpose and scope

This document describes the current browser-to-server flow in SentinelPay, from signing in through signing out. SentinelPay is a Django demonstration application: its account data is synthetic and payments use a deterministic local simulator. It does not connect to a bank or move real money.

Application entry points

- Django starts through `manage.py` and loads settings from `sentinelpay/settings.py`.
- `sentinelpay/urls.py` routes the sign-in and sign-out endpoints, the customer overview, Django administration, and payment routes.
- Most customer pages require an authenticated Django user. Fraud operations additionally require a staff/superuser account or membership in the `Fraud Analysts` group.
- Django templates under `templates/` render the pages; `static/css/site.css` supplies the shared styles and CSS-only animations.

End-to-end user journey

1. Open the application

The visitor opens `/`. The dashboard view is protected by Django's `login_required` decorator. If no authenticated session exists, Django redirects the visitor to `/login/`.

2. Sign in

The sign-in page uses Django's built-in `LoginView` with the SentinelPay login template. The browser submits the username and password over a CSRF-protected POST. Django checks the credentials and creates an authenticated session. Successful sign-in redirects to the configured dashboard (`LOGIN_REDIRECT_URL`); invalid credentials return the sign-in page with an error.

There is no public registration or password-reset workflow in the current routes. User accounts are managed by Django administration or project setup.

3. View the customer overview

The dashboard queries only records associated with the signed-in customer:

- Their own customer profile, bank accounts, and available/ledger balances.
- Their payment counts, monthly successful spend, six-month payment activity, and recent transactions.
- A count of payments requiring attention.

Customer numbers are generated synthetic references. Name, email, phone, date of birth, and address are displayed from profile data only; missing values remain "Not provided." KYC defaults to "Details not provided" and customer risk to "Not assessed." The charts are rendered from database results and are not an independent source of transaction data. Account references are masked in the customer-facing view.

4. Edit customer details

The customer opens `/profile/` from the workspace navigation. The form allows the signed-in customer to update their name, email, Indian mobile number, date of birth, and address. The server validates the mobile format and rejects a future date of birth. A successful save updates the linked customer profile and the corresponding Django user name/email fields. KYC and risk classifications are system-managed and cannot be set from the customer form. Aadhaar and PAN are not collected.

5. Start a payment

The customer opens `/payments/new/`. The form offers:

- Source accounts owned by that customer.
- That customer's own beneficiaries and project-wide beneficiaries.

- A positive decimal amount and one of the configured simulator routes.

- A generated idempotency key retained as a hidden form field.

Django validates the submitted values against server-side form fields and querysets. Only active accounts and active eligible beneficiaries are offered; the service checks these conditions again at confirmation. Another customer's private account or beneficiary is not an eligible choice. A valid form does not create or debit a payment: it stores a short-lived payment draft in the customer's session and redirects to review.

6. Review the payment

At `/payments/review/`, the server reconstructs and validates the form from the session draft. If the saved details are missing or no longer valid, the draft is removed and the customer is asked to restart. The review shows the amount, source account, recipient, masked beneficiary destination when present, and the selected simulator route.

The customer may edit the details, cancel, or confirm. Cancelling removes the draft without creating a payment. Confirming sends a CSRF-protected POST. The review page does not decide the fraud result or payment outcome.

7. Server-side processing and fraud evaluation

On confirmation, `payments.views.payment_review` calls `payments.services.create_payment`. The service performs the authoritative checks and processing:

1. Validates the amount and configured merchant code.
2. Creates a request fingerprint and checks the customer's idempotency key. An identical retry returns the original payment; a different request using the same key is rejected.
3. Within a database transaction, looks up the source account while verifying ownership.
4. Looks up the beneficiary while allowing only the customer's own or project-wide beneficiary.
5. Checks the available account balance.
6. Builds a fraud context from persisted payment history, the beneficiary creation time, amount, and local time.
7. Calls the existing deterministic rules engine in `fraud_engine/engine.py`.
8. Persists the payment with the evaluated score, decision, triggered-rule explanations, and ruleset version, and appends processing events through the audit service.

The eight rule results and the decision thresholds are documented in the project README. They are educational rule points, not calibrated fraud probabilities. Device and IP-country rules are inactive because the application does not collect those signals.

8. Apply the fraud decision and simulator outcome

The backend enforces the result before submission:

- `DECLINE` becomes `REJECTED`; no simulator request or ledger debit is made.

- `REVIEW` and `STEP_UP` become `HELD_FOR_REVIEW`; no debit is made and a fraud case is created. The step-up challenge flow is not implemented.

- An approved payment becomes `SUBMITTED` and is passed to `payments.gateway.SimulatedGateway`. The simulator's selected route determines the outcome.

Simulator route	Recorded outcome	Debit behavior
`SIM-GROCERY`	`SUCCEEDED`	A ledger debit is posted once if the balance update succeeds.
`SIM-DECLINE`	`FAILED`	No debit.
`SIM-TIMEOUT`	`UNKNOWN`	No debit; no automatic retry or reconciliation workflow.
`SIM-ERROR`	`FAILED`	No debit.

All status changes use the state transition function in `payments/states.py`. In the same database transaction, the service creates a `banking_transaction` record tied to the payment, account, and optional beneficiary. Its status and risk score copy the actual persisted payment result. A successful simulator result decrements both available and ledger balances and creates the existing ledger debit; non-success outcomes do not change either balance. A timeout remains unresolved with no processed timestamp or debit.

9. View the result

After processing, the browser is redirected to the payment receipt identified by a unique public reference. The receipt is scoped to the signed-in customer and displays the persisted payment status and transaction reference, amount, account, recipient, fraud score and decision, triggered rules, recorded audit timeline, and any ledger entry. The dashboard and transaction history show the same persisted transaction reference and result.

The timeline is read from stored audit events. Its animation is a presentation effect; it does not imply that the transaction is still processing. The risk meter uses the stored score rather than inventing a new value.

10. Review fraud cases (authorized users only)

Staff, superusers, and members of the `Fraud Analysts` group can open `/payments/fraud/` and `/payments/fraud/cases/<id>/`. The monitoring page queries persisted payment and case records. Authorized reviewers may update a case through allowed status transitions and add a note. Each accepted change records the reviewer, old and new statuses, note, timestamp, and payment correlation ID in the append-only audit log.

Customer users who do not meet the server-side permission check receive HTTP 403 even if they manually enter a fraud route.

11. Sign out

The shared navigation submits a CSRF-protected POST to `/logout/`. Django's built-in `LogoutView` ends the authenticated session and redirects according to Django's logout behavior. The user must sign in again to access protected pages.

Data and configuration flow

- SQLite is the default database, stored at the project root as `db.sqlite3`.
- Setting `DB_ENGINE=mysql` switches settings to the configured MySQL environment variables. MySQL is optional and has not been verified as part of the current setup.
- Credentials and settings are read from process environment variables. `.env.example` contains placeholders; Django does not automatically load a `.env` file.
- `python manage.py migrate` applies schema migrations. This document does not imply that a migration has already been applied to a particular local database.
- The banking schema migration backfills customer profiles from names/emails already stored in Django users and transaction records from existing payments. It preserves the existing available balance and initializes ledger balance to that value as a cutover snapshot; it cannot reconstruct historical opening balances or ledger balances.

Current flow limitations

- Payments are simulated; no real payment network or provider is contacted.
- There is no public registration, customer control of KYC/risk classifications, live transaction feed, or external fraud API.
- Unknown gateway outcomes remain unresolved until a future reconciliation feature exists.
- Payment limits, fees, step-up authentication, and customer-managed beneficiary creation are not implemented.